

❖ 알고리즘 성능 분석 방법

■ 공간 복잡도

- 알고리즘을 프로그램으로 실행하여 완료하기까지 필요한 총 저장 공간량
- $\text{공간 복잡도} = \text{고정 공간} + \text{가변 공간}$

■ 시간 복잡도

- 알고리즘을 프로그램으로 실행하여 완료하기까지의 총 소요시간
- $\text{시간 복잡도} = \text{컴파일 시간} + \text{실행 시간}$
 - 컴파일 시간 : 프로그램마다 거의 고정적인 시간 소요
 - 실행 시간 : 컴퓨터의 성능에 따라 달라질 수 있으므로 실제 실행시간보다 명령문의 실행 빈도수에 따라 계산
- 실행 빈도수의 계산
 - 지정문, 조건문, 반복문 내의 제어문과 반환문은 실행시간 차이가 거의 없으므로 하나의 단위시간으로 갖는 기본 명령문으로 취급



■ 피보나치 수열 알고리즘의 빈도수 구하기

fibonacci(n)

[알고리즘 2-1]

```
01  if (n<0) then
02      stop ;
03  if (n≤1) then
04      return n ;
05  f1 ← 0 ;
06  f2 ← 1 ;
07  for (i←2 ; i≤n ; i←i+1) do {
08      fn←f1+f2 ;
09      f1←f2 ;
10      f2←fn ;
11  }
12  return fn ;
13 end
```

- $n < 0$, $n = 0$, $n = 1$ 의 경우에 대한 실행 빈도수
 - for 반복문이 수행되지 않기 때문에 실행 빈도수가 작다.

행 번호	$n < 0$	$n = 0$	$n = 1$
1	1	1	1
2	1	0	0
3	0	1	1
4	0	1	1
5~13	0	0	0

- $n > 1$ 의 일반적인 경우에 대한 실행 빈도수

➤ n 에 따라 for 반복문 수행

행 번호	실행 빈도수	행 번호	실행 빈도수
1	1	8	$n-1$
2	0	9	$n-1$
3	1	10	$n-1$
4	0	11	0
5	1	12	1
6	1	13	0
7	n		

➤ 총 실행 빈도수

$$= 1+0+1+0+1+1+n+(n-1)+(n-1)+(n-1)+0+1+0 = 4n+2$$

■ 시간 복잡도 표기법

- 빅-오(Big-Oh) 표기법 사용
- 빅-오(Big-Oh) 표기법 순서
 - ① 실행 빈도수를 구하여 실행시간 함수 찾기
 - ② 실행시간 함수의 값에 가장 큰 영향을 주는 **n에 대한 항을** 선택하여
 - ③ 계수는 생략하고 **O** (Big-Oh)의 오른쪽 괄호 안에 표시
- 피보나치 수열의 시간 복잡도 = $O(n)$
 - ① 실행시간 함수 : $4n+2$
 - ② n에 대한 항을 선택 : $4n$
 - ③ 계수 4는 생략하고 **O** (Big-Oh)의 오른쪽 괄호 안에 표시 : $O(n)$

■ 빅-오(Big-Oh) 표기법 (참고자료)

- 자료의 개수가 많은 경우에는 차수가 가장 큰 항이 가장 영향을 크게 미치고 다른 항들은 상대적으로 무시될 수 있다.

$n=1000$ 인 경우

$$T(n) = n^2 + n + 1$$

99%

1%

- (예) $n=1,000$ 일 때, $T(n)$ 의 값은 1,001,001이고 이중에서 첫 번째 항인 n^2 의 값이 전체의 약 99%인 1,000,000이고 두 번째 항인 n 의 값이 1000으로 전체의 약 1%를 차지한다.
- 따라서 보통 시간복잡도 함수에서 가장 영향을 크게 미치는 항만을 고려하면 충분하다. → **$O(n^2)$**

■ 빅-오(Big-Oh) 표기법 (참고자료)

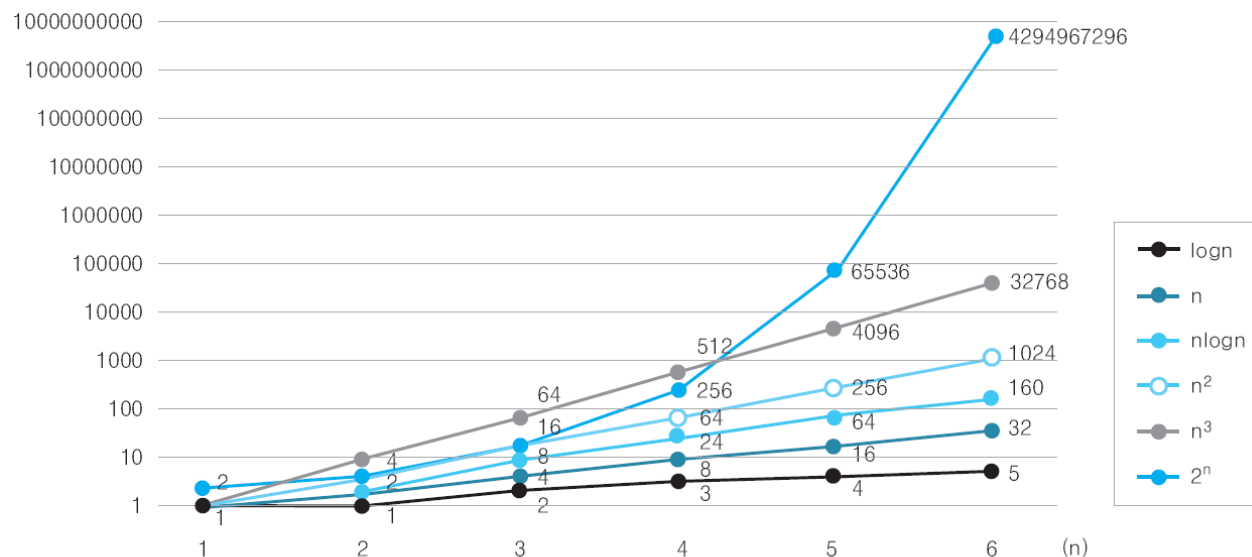
- 빅오는 함수의 상한을 표시한다.
 - (예) $n \geq 5$ 이면 $2n+1 < 10n$ 이므로 $2n+1 = O(n)$

예제 1.1 빅오 표기법

- $f(n) = 5$ 이면 $O(1)$ 이다. 왜냐하면 $n_0 = 1$, $c = 10$ 일 때, $n \geq 1$ 에 대하여 $5 \leq 10 \cdot 1$ 이 되기 때문이다.
- $f(n) = 2n+1$ 이면 $O(n)$ 이다. 왜냐하면 $n_0 = 2$, $c = 3$ 일 때, $n \geq 2$ 에 대하여 $2n+1 \leq 3n$ 이 되기 때문이다.
- $f(n) = 3n^2+100$ 이면 $O(n^2)$ 이다. 왜냐하면 $n_0 = 100$, $c = 5$ 일 때, $n \geq 100$ 에 대하여 $3n^2+100 \leq 5n^2$ 이 되기 때문이다.
- $f(n) = 5 \cdot 2^n + 10n^2 + 100$ 이면 $O(2^n)$ 이다. 왜냐하면 $n_0 = 1000$, $c = 10$ 일 때, $n \geq 1000$ 에 대하여 $5 \cdot 2^n + 10n^2 + 100 \leq 10 \cdot 2^n$ 이 되기 때문이다.

■ 각 실행 시간 함수에서 n값의 변화에 따른 실행 빈도수 비교

logn	<	n	<	nlogn	<	n ²	<	n ³	<	2 ⁿ
0		1		0		1		1		2
1		2		2		4		8		4
2		4		8		16		64		16
3		8		24		64		512		256
4		16		64		256		4096		65536
5		32		160		1024		32768		4294967296



[그림 2-17] n값에 대한 실행 시간 함수 그래프